

Вебинар · AI-агенты и оркестрация

МСР: интеграции по стандарту



Кирилл Кухарев

Старший AI Engineer, Raft

План вебинара

01

MCP vs «просто REST»

Один слой для инструментов: кто их описывает, кто разрешает вызов.

02

LangFlow

Быстро доказать сценарий на канве и понять, где пора в репозиторий.

03

LangGraph

Код как договор с эксплуатацией: паузы, повторы, устойчивые шаги.

04

Продакшен-«клей»

Событие из мира будит цепочку: конструкторы и облака как хосты.

05

Риски и наблюдаемость

Что модель не должна нажимать, что измерять, чтобы улучшать.

06

Демо-сценарий

Контракт с данными: выборка, ограничения вывода, граница агрегатов и текста.

07

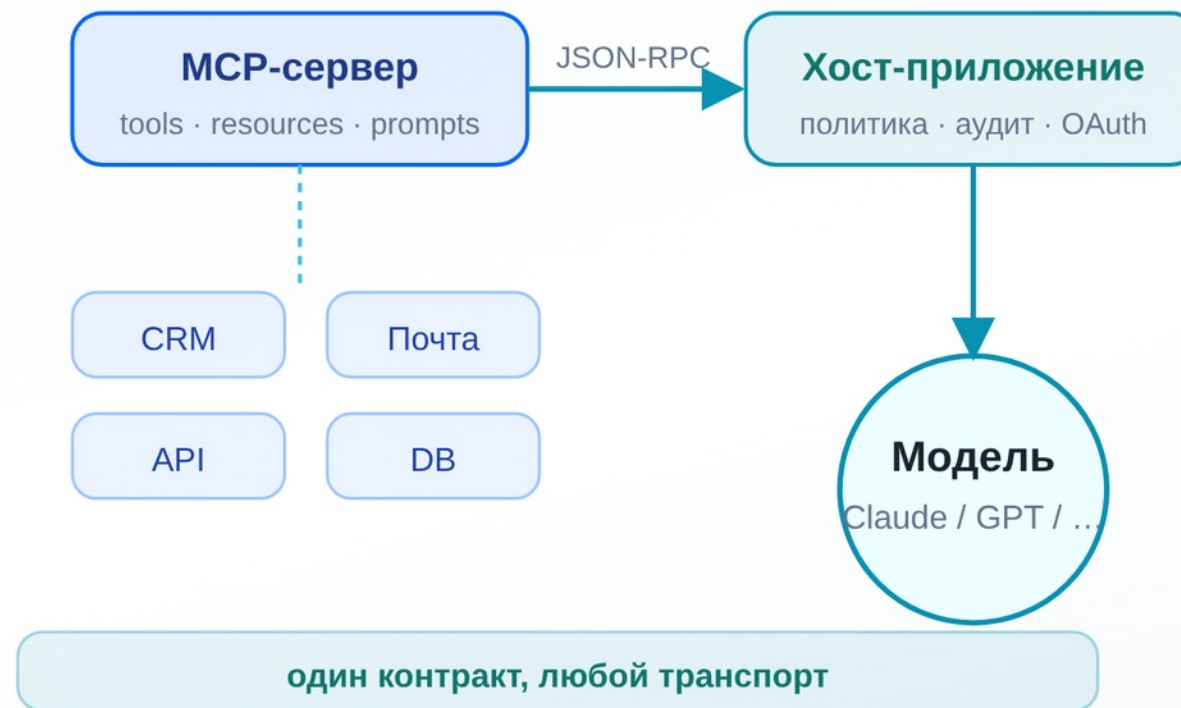
Материалы и источники

Спецификация MCP и ссылки для углубления — в конце.

БЕЗ ОБЩЕГО ДОГОВОРА (N×M)

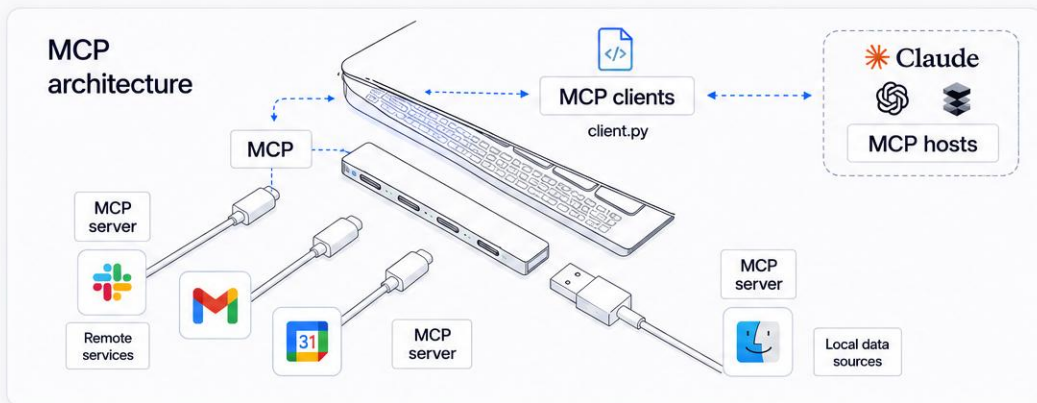


СО СЛОЕМ MCP (N+M)



MCP vs «просто REST»

Один слой для инструментов: кто их описывает, кто разрешает вызов. Не «ещё один API ради API», а общий язык между моделью и инструментами — что доступно, как спросить и кто имеет право нажать кнопку.



СЛОЙ СООБЩЕНИЙ

Один контракт, но разные транспорты

Везде один и тот же протокол (JSON-RPC): те же методы и локально в процессе, и по сети — в том числе по HTTP со стримом. Описание одно, поэтому смена клиента не превращается в «другой API с сюрпризами».

РАЗДЕЛЕНИЕ РОЛЕЙ

Сервер говорит «что есть», хост «что можно»

MCP-сервер публикует инструменты, ресурсы и промпты. Хост в вашем приложении решает, что можно вызывать, с какими параметрами и что писать в журнал — в проде без этого не обойтись.

ПРАКТИЧЕСКИЙ СМЫСЛ

Один раз подключили — много где используете

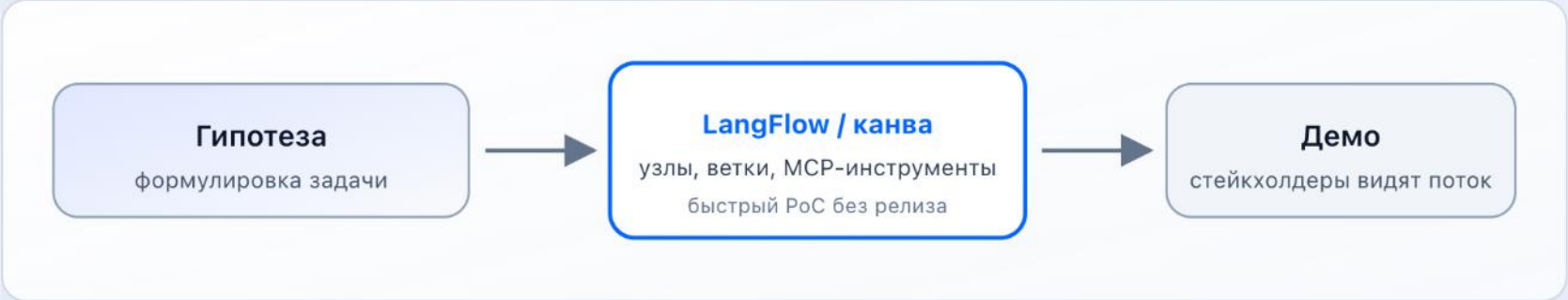
Один и тот же сервер подключают к IDE, десктопному агенту, LangFlow или своему раннеру — без отдельных обёрток и «ещё одного Swagger» под каждый продукт.

Канва vs репозиторий: где удобно, а где опасно

Сначала снимают риск дешёвыми средствами — гипотезы и демо на канве, в LangFlow. Потом закрепляют контур в коде: ревью, тесты, наблюдаемость, там сбой уже как инцидент в проде.



ЭТАП 1
Канва — доказательство сценария



- На канве дешевле **перебрать три варианта** цепочки и показать «как могло бы работать» за часы, а не за спринт в репозитории.
- Удобно подключать внешние MCP и HTTP — **сценарий живёт визуально**, обсуждение идёт по схеме, а не по диффам.
- **Риск:** без экспорта в код и без контрактов с данными легко зафиксировать «красивую игрушку», которую нельзя воспроизвести в проде один в один.

Канва vs репозиторий: где удобно, а где опасно

Сначала снимают риск дешёвыми средствами — гипотезы и демо на канве, в LangFlow. Потом закрепляют контур в коде: ревью, тесты, наблюдаемость, там сбой уже как инцидент в проде.

01

Канва

02

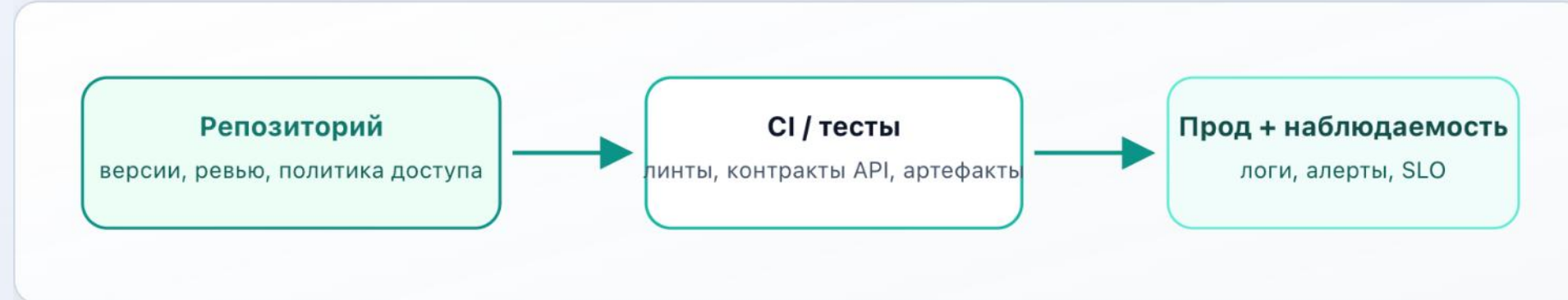
Контур

03

Ускоритель

ЭТАП 2

Контур — где ошибка стоит денег



- Источник истины для денег и регуляtorики — **git, пайплайн, секреты**, а не «последняя сохранённая вкладка» в браузере.
- Здесь вы фиксируете **идемпотентность, ретраи, границы доверия к MCP** и кто подписывает опасные действия.
- **Риск:** если канва и репо расходятся, команда спорит о двух правдах — нужна явная миграция сценария в код или в спецификацию.

Канва vs репозиторий: где удобно, а где опасно

Сначала снимают риск дешёвыми средствами — гипотезы и демо на канве, в LangFlow. Потом закрепляют контур в коде: ревью, тесты, наблюдаемость, там сбой уже как инцидент в проде.

01

Канва

02

Контур

03

Ускоритель

ЭТАП 3

Ускоритель — мост между канвой и кодом



- LangFlow — **ускоритель**: быстрее собрать цепочку, опубликовать как MCP или вынести узлы в сервис без переписывания всего с нуля.
- Договоритесь о правиле: **что считается «готово»** только после появления в репо (тест, спека, OpenAPI) — тогда канва не подменяет прод.
- **Сигнал «пора в код»**: ветвления не помещаются на экране, нужны долгие саги, строгий аудит шагов или единый контракт для нескольких команд.

Визуальный слой: скорость гипотез и зона ответственности

LangFlow снимает ощущение «всё надо кодить» — ускоритель гипотез и демо для стейкхолдеров. Но у канвы есть естественный потолок.

✦ Когда канва — ваш друг

Быстро показать «как могло бы работать», не тратя неделю на репозиторий.

- PoC цепочки: чат → HTTP к данным → несколько шагов с моделью.
- Подмешать внешние MCP-серверы узлами MCP Tools / MCP Client.
- Опубликовать flow как MCP Server — один инструмент для IDE и других хостов.
- Демо стейкхолдерам без кода — понятная визуальная схема.

⚠ Когда пора переезжать в код

Там, где нужны предсказуемость, долгая память пайплайна и строгий CI.

- Сложные ветвления, многоступенчатые саги, юнит-тесты как часть DoD.
- Командный git: версии JSON-flow и ревью диффов требуют дисциплины.
- Большие ответы API — сужайте контекст до модели до входа в LLM.
- Продакшен с SLA: нужны retry, мониторинг, трассировка.

Код как договор с эксплуатацией: паузы, повторы, устойчивые шаги

Источник истины для продакшна: систему можно отлаживать, версионировать и возобновлять после сбоя. Четыре опорных понятия являются «скелетом» надёжного агента.

[1]

Граф состояний

Явные узлы и переходы: условные рёбра и подграфы помогают собирать сложные процессы из кусков, а не из одной простыни промпта.

[2]

Checkpointing

Снимки по шагам и поток `thread_id` : можно откатиться, продолжить с середины или отладить «что модель решила на этом шаге».

[3]

Человек в контуре

`interrupt()` и возобновление через `Command(resume=...)` — паттерн «опасное действие ждёт согласования», без костылей вне очереди.

[4]

Durable execution

Длинные паузы (ожидание человека, внешнего API) не должны терять прогресс. Там, где критично — держите детерминизм на ключевых переходах.

Граф как контракт между кодом и поддержкой



Повтор

Что можно откатить и прогнать снова без сюрприза



Пауза

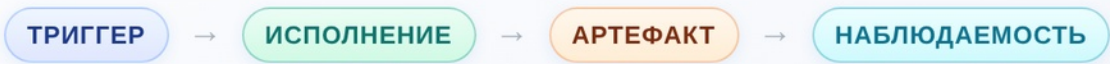
Граф явно ждёт человека — не «зависло», а контракт



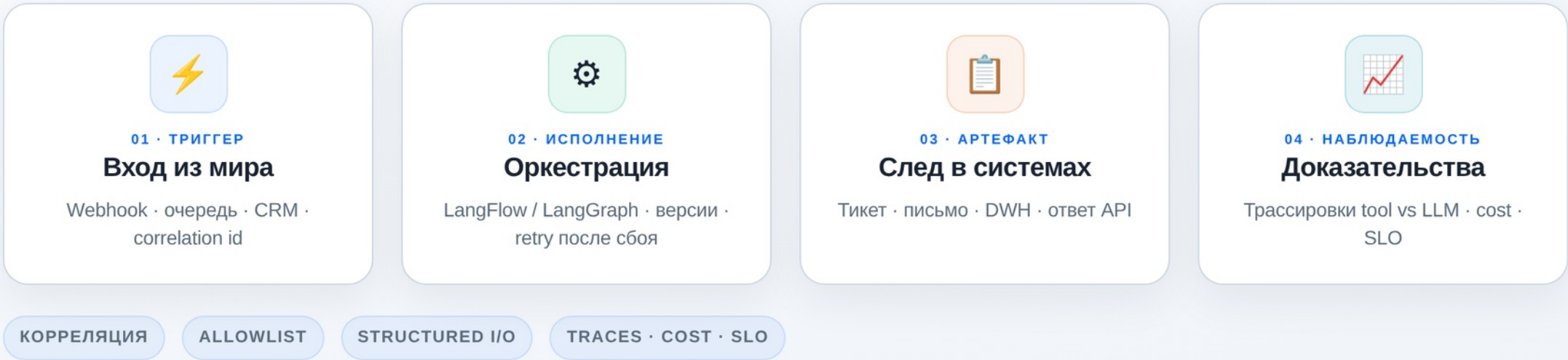
Шаг

Ни один обязательный переход не теряется в длинном ране

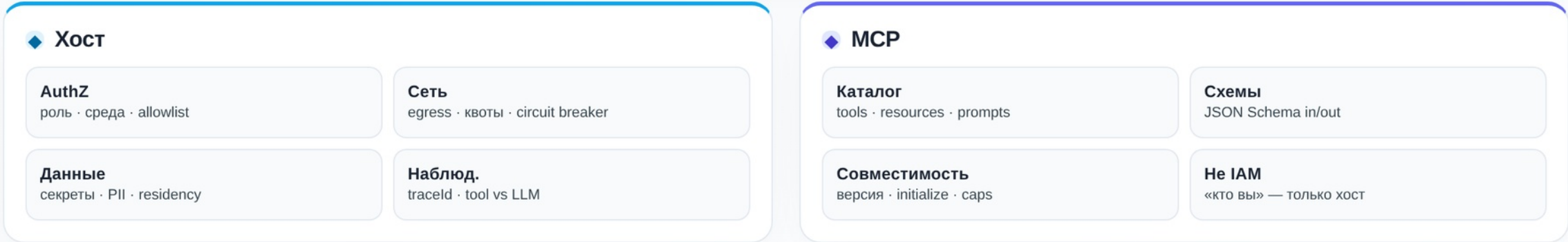
Событие из мира будит цепочку



Политика и рантайм — на хосте. MCP задаёт контракт инструментов до того, как модель «нажмёт кнопку».



Хост и MCP — две зоны



Где крутится агент и как стыкуется MCP

ПЛОЩАДКА	ТИПИЧНАЯ РОЛЬ	ЧТО ЗАФИКСИРОВАТЬ ДО ПРОДАКШЕНА
Dify	Приложения на базе LLM, RAG, публикация flow	• Редакция OSS или Cloud и кто инициирует исходящий вызов MCP • Нужен ли обратный контур «ваше приложение как MCP server» • Матрица прав: кто из ролей может дернуть какой инструмент
n8n	Интеграции, webhooks, AI Agent (LangChain)	• Где лежат токены: Credential Manager, vault, переменные среды • MCP через HTTP или прокси: граница сети и allowlist исходящих хостов • Как фиксируется версия контракта при обновлении нод
AWS Bedrock	Корпоративные агенты, AgentCore	• Identity для вызовов и MCP proxy в AgentCore • VPC endpoints и маршрут трафика к MCP • Лимиты вызовов и бюджет на шаг в design doc
Microsoft Copilot Studio	Процессы в контуре M365	• Каталог коннекторов и классификация данных по чувствительности • Что может покинуть тенант в сторону внешнего MCP • Что остаётся внутри облака Microsoft
Flowise	Визуальные цепочки LangChain	• Есть ли стабильная MCP интеграция в вашей мажорной версии • План обновлений при смене major у зависимостей • Кто владеет конфигом endpoints в проде
Langflow	Визуальные потоки и агенты на базе LangChain	• Узел MCP в графе: кто разрешён на побочные эффекты и куда уходит сеть • Воспроизводимый экспорт flow и закрепление версий зависимостей • Разводка dev и prod по credentials и исходящим хостам

Дороже галлюцинации: несанкционированное действие

Неверный текст правят промптом и офлайн оценками. Неверный вызов инструмента уже инцидент в проде.

Каталог MCP без политики хоста превращается в панель кнопок без охранника. Нужны явный allowlist, владелец секретов и изоляция недоверенного контента от системных инструкций.

ПОЛИТИКА

Закрытый перечень инструментов и параметров до генерации. Произвольный веб как инструмент только после ревью.

СЕКРЕТЫ

Endpoint MCP с токеном считается секретом приложения. Vault, ротация, сетевые ACL. Секреты не кладём в промпт.

ДАННЫЕ И АУДИТ

Разные учётки для внешнего API и внутренних CRM. Минимальные права. Раздельные журналы для шагов модели и для вызовов инструментов.

КОНТЕНТ

Системный промпт и данные из внешних источников разводим по каналам. Снижаем риск подмены инструкций через вход пользователя.

Что модель не должна нажимать

Исследования 2025: 84.2% успех tool-poisoning при auto-approve и у 43% публичных серверов были уязвимости command injection.

**Host-policy обязателен**

Host должен решать список разрешённых tools и параметры до вызова; логировать request/response в обезличенном виде.

**Секреты в vault, не в URL**

URL MCP-сервера с секретами — как API key: только vault, rotation, ограничение по сети и IP.

**Least privilege для сервисов**

Разделяйте промышленный доступ: принцип наименьших привилегий для service accounts и OAuth scopes.

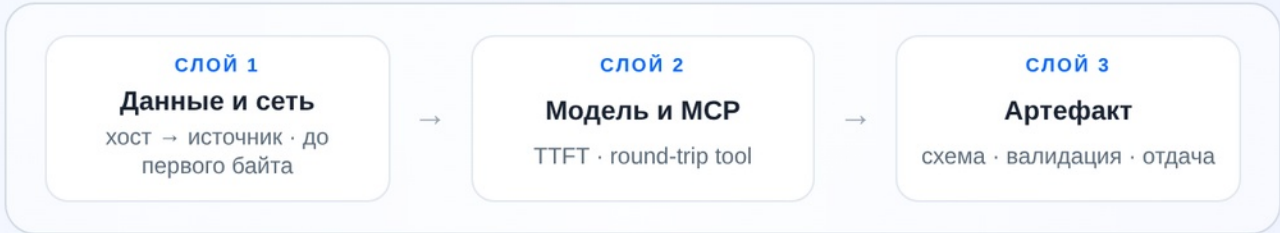
**Prompt injection isolation**

Не смешивайте необработанный веб-контент с системными инструкциями без изоляции — ключевая атака на MCP-агентов.

Топ угроз по данным исследований 2025 (% инцидентов)

Предсказуемость цикла

Продуктовый сигнал — не оценка «красоты ответа», а **воспроизводимый путь** от входа до артефакта: отдельные бюджеты на данные, модель, tools и формат выхода. Сумма средних по слоям не заменяет сквозной SLO.



Сеть / данные
Латентность и ошибки до вызова модели: API, БД, кэш.
Без этого TTFT и round-trip «улучшают» не то место.

TTFT
Первый токен ответа — отдельно от полного generation и от времени tool; иначе смешиваете бюджеты.

Round-trip
Сквозной путь «запрос пользователя → готовый результат» — то, что обычно фиксируют в P95/P99 для UX.

Токены / шаг
Стоимость и лимит на один шаг графа; ловите дрейф после смены промпта, набора tools или контекста.

JSON и правки
Доля валидного structured output и ручных правок — измеримая «терпимость» пайплайна к формату, не субъективная оценка.

HITL
Доля шагов с человеком в контуре; в prod задают целевой потолок и причину (риск, качество, регуляторика).

ВХОДНАЯ ВЫБОРКА
Объём, смещения и пустые cohorts фиксируйте артефактом рядом с пайплайном. Без явной выборки оптимизация метрик часто усиливает случайный шум, а не продукт.

Метрики агентного MVP

- Латентность по сегментам: HTTP к данным vs TTFT модели vs полный пайплайн — сравниваете с baseline.
- Стоимость токенов на шаг и доля повторных прогонов при ошибках JSON.
- Успешность structured output и частота ручной правки (HITL в LangGraph для «опасных» действий).
- Качество данных: объём выборки, покрытие salary, региональные искажения.

P95

ЛАТЕНТНОСТЬ
< 3 500 мс

98%

JSON SUCCESS
structured output

\$0.04

СТОИМОСТЬ / ШАГ
включая tools

HITL

ОПАСНЫЕ ДЕЙСТВИЯ
interrupt() паттерн

Честный контракт с данными: выборка, handoff и граница вывода

Демонстрация не претендует на репрезентативность рынка труда — в фокусе воспроизводимый контракт между шагами: сжатая выборка вакансий, явные ограничения вывода и граница между агрегатами и текстом для человека.

ВЫБОРКА

Параметры запроса и объём фиксируются в артефакте; пустые выборки — отдельная ветка с fallback или офлайн-файлом.

HANDOFF

Между узлами графа — один схемой заданный артефакт; проще тестировать, версионировать и дебажить.

ГРАНИЦА

Таблицы и числа — воспроизводимая аналитика; мотивационный текст резюме — отдельный слой с явной пометкой источника.

Спецификация MCP и ссылки для углубления

Спецификация MCP, документация LangFlow/LangGraph, платформы, hh API и облачные рантаймы — для самостоятельного углубления.

1	Model Context Protocol — спецификация	modelcontextprotocol.io/specification/latest
2	MCP Transports	modelcontextprotocol.io/docs/concepts/transports
3	LangFlow MCP Server	docs.langflow.org/mcp-server
4	LangFlow MCP Tools	docs.langflow.org/mcp-tools
5	LangFlow MCP Client	docs.langflow.org/mcp-client
6	LangFlow API Request	docs.langflow.org/api-request
7	LangGraph persistence · interrupts · durable execution	Persistence Interrupts (HITL) Durable execution
8	Dify MCP	docs.dify.ai — MCP Tools dify.ai — релиз v1.6 (two-way MCP)
9	n8n AI Agent	docs.n8n.io — AI Tools Agent blog.n8n.io — Build AI agent
10	AWS Bedrock MCP / AgentCore	AWS Blog — Bedrock Agents + MCP AgentCore — MCP Runtime
11	Google Agent Builder	cloud.google.com/products/agent-builder
12	Microsoft Copilot Studio	learn.microsoft.com/microsoft-copilot-studio

Спасибо

Вопросы, разбор ваших кейсов и возможных паттернов
деплойа.